

This is a compilation of small tutorials that are useful for beginners nonetheless!

Contents

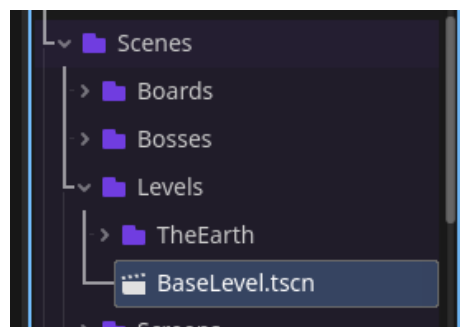
How to create a new level.....	1
How to create a new enemy.....	9
How to create a new character state.....	20

How to create a new level

First of all, how to create new levels. If you're not a beginner, you'll probably know it's simple: you just create an inherited scene from the base level and work from there. But I'm writing this tutorial for people who aren't familiar with Godot yet.

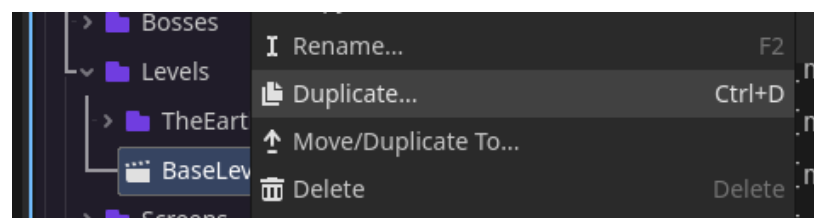
I've already mentioned that you'll need to create an inherited scene from the base level. But where is that "base level" located?

It's in "Scenes/Levels" folder:



Okay, now what's an "inherited scene"?

An inherited scene is basically when you take an existing scene with all its objects/nodes and create a new scene out of it. You might think it's the same as duplicating a scene like this:

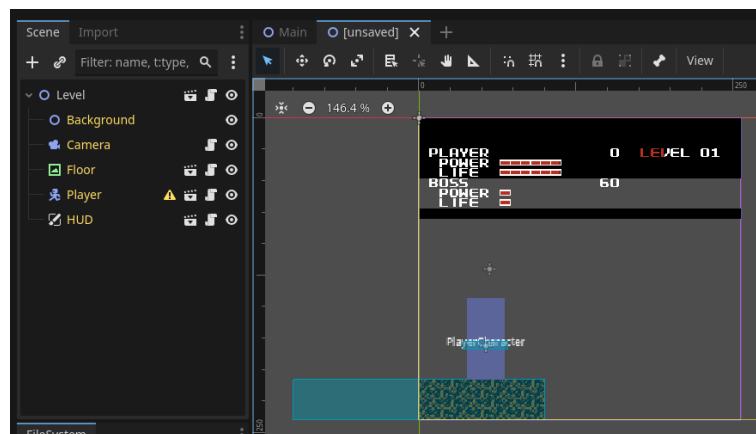
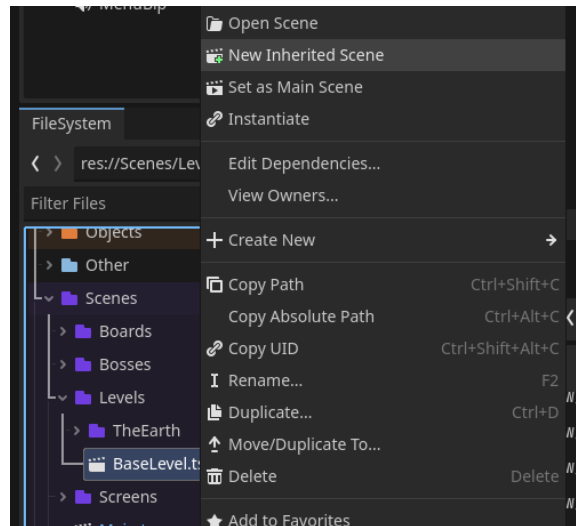


And yes, duplicating a scene and creating an inherited scene from it are pretty similar and you get seemingly the same result at first, but there's an important difference: when you change the base scene, an inherited scene changes accordingly.

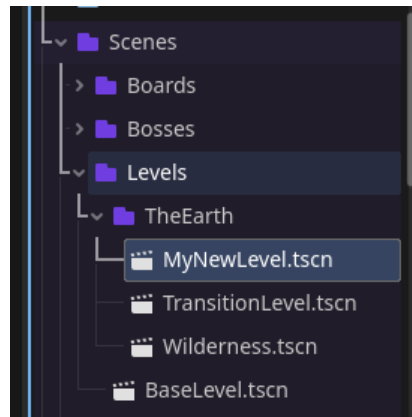
For example, you already have a lot of levels, and you decided to replace the HUD object with your own without affecting the original HUD object scene in the project, you can delete the original HUD object and add your own, and it will be added to all of the levels in your game! How cool is that?

Okay, now I think we can actually start creating a new level.

Right click on the base level scene and select “New Inherited Scene”.

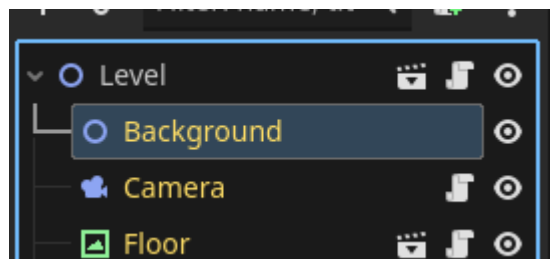


Now save it somewhere in your project, preferably in the folder that represents the planet you’re making this level for. If it’s for the Earth, put the level into “Scenes/Levels/TheEarth” folder, if it’s going to be used in a different planet, put it into “Scenes/Levels/<your planet name here>” folder.

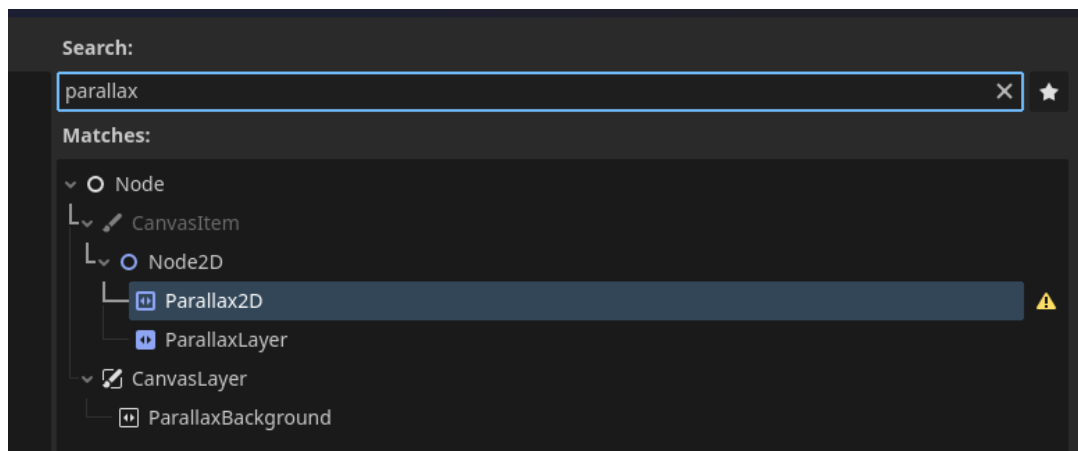


Nice, we have a new empty level. Let's create a simple background.

You may notice that there's a Node2D that's called "Background" in the scene:

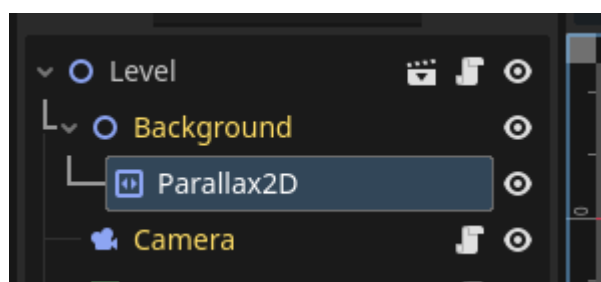


That's where we add our Parallax2D nodes.



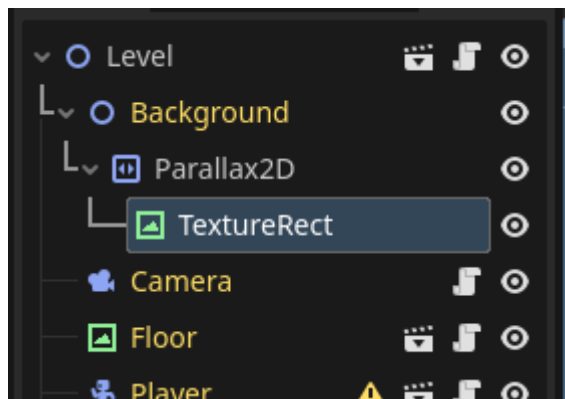
The difference between Parallax2D and ParallaxLayer is that Parallax2D is the newer solution that's more flexible I think, haven't checked those in quite some time but Parallax2D is definitely newer version of ParallaxLayer. And ParallaxBackground was used with ParallaxLayer.

We now have a parallax node:

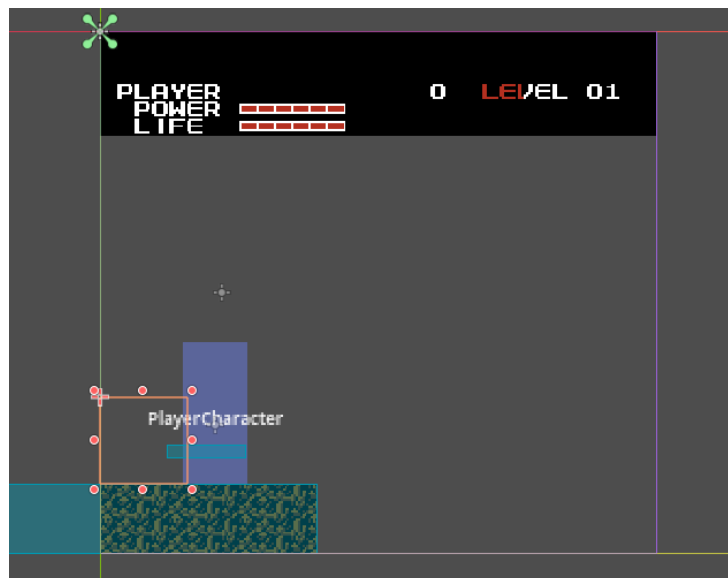


Let's make it show something. You can use any node that can be displayed, I will use TextureRect for this tutorial but something like Sprite2D can also work.

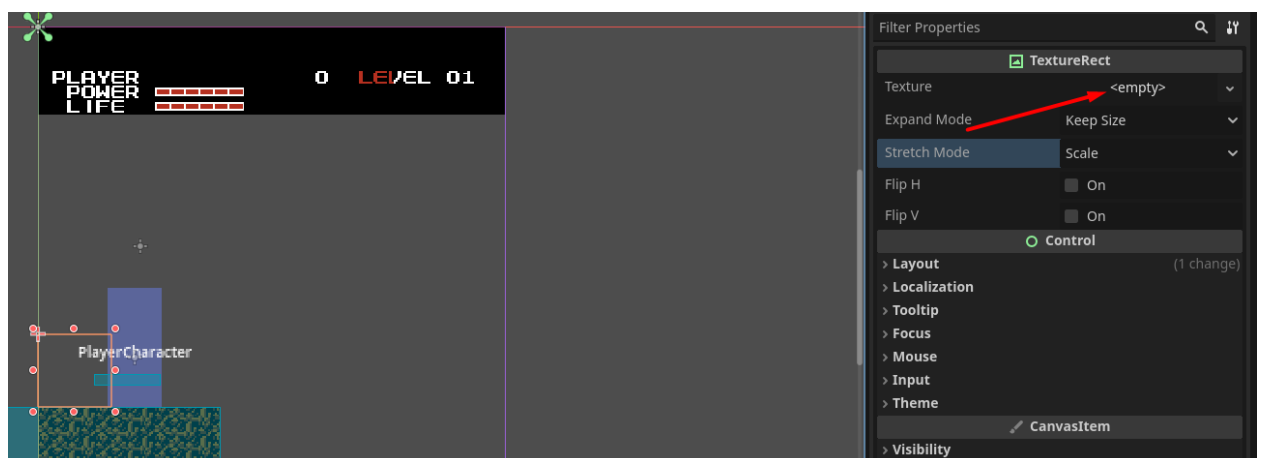
So first, I added the TextureRect node to the Parallax2D node as a child node.



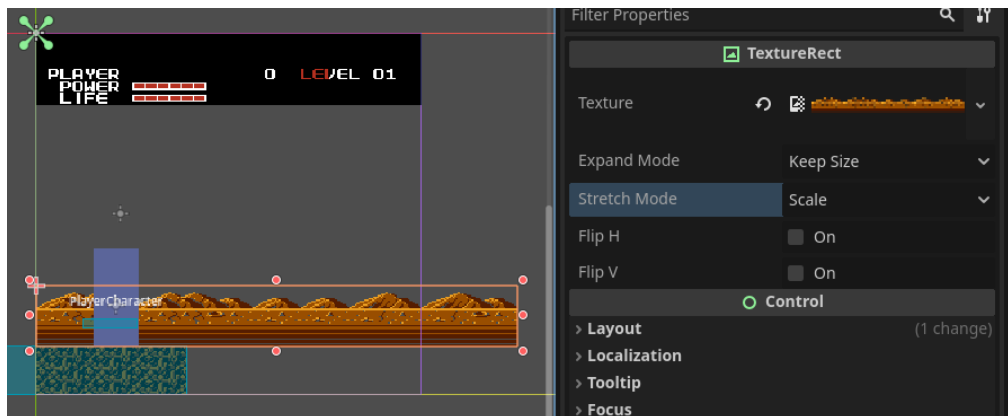
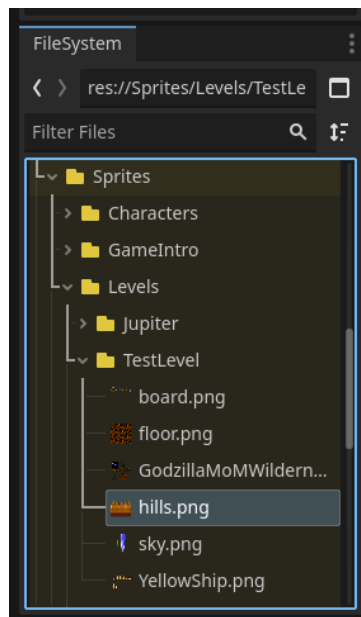
Then I moved the TextureRect node where I want to have my background.



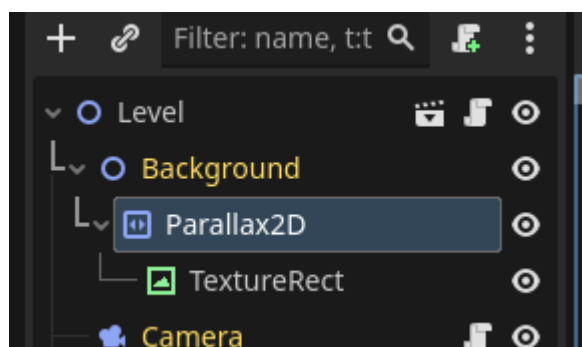
Now I need to add a texture to it.



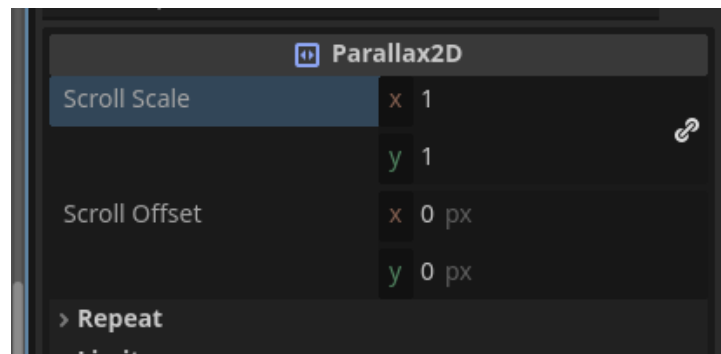
I will drag and drop the existing Wilderness background just as an example.



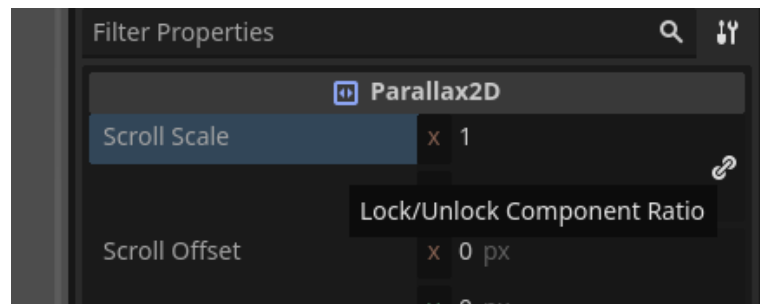
If you test the level now (check out “How to create a new character” tutorial, section “Testing our new character”) you will notice that this background piece has the same speed as the floor and it doesn’t repeat, so we need to change that. Select the Parallax2D node:



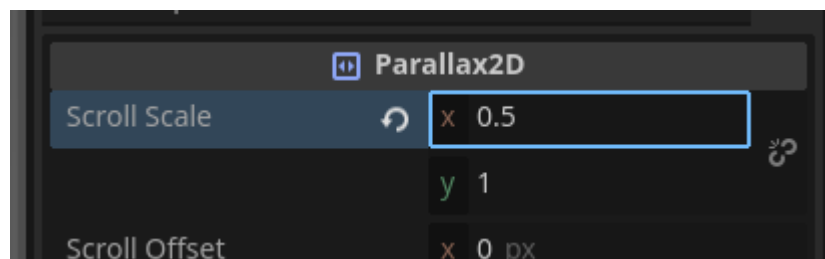
And here we need to change the scroll speed property:



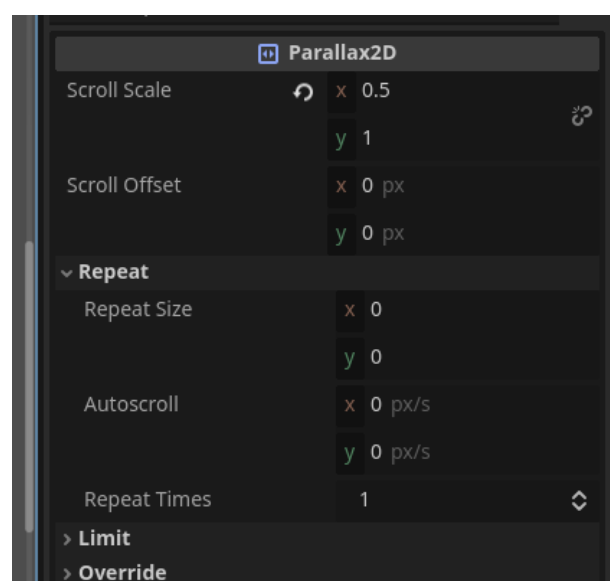
You may also want to untick this:



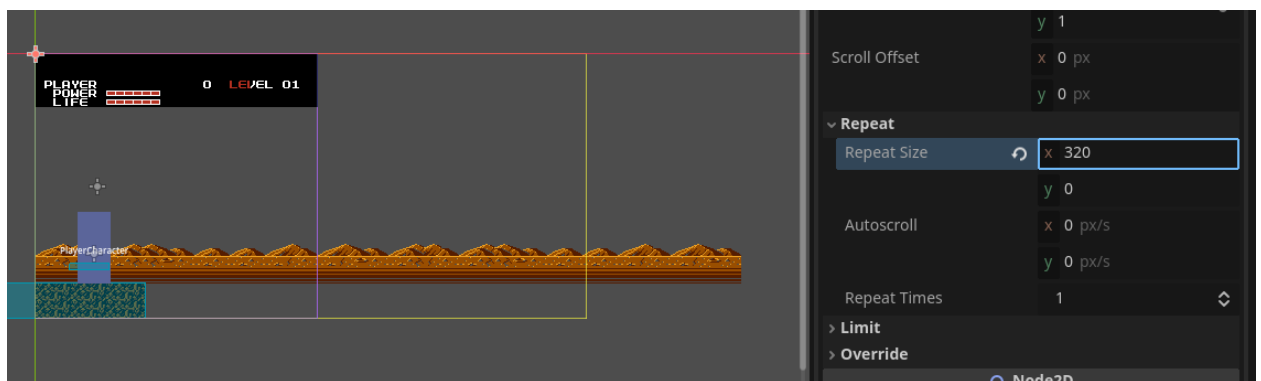
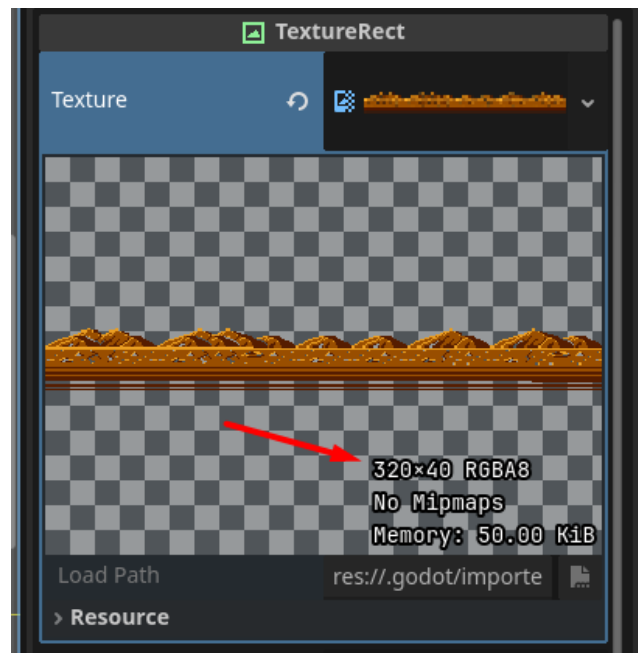
Now you can try something like 0.5 for Scroll Scale X value.



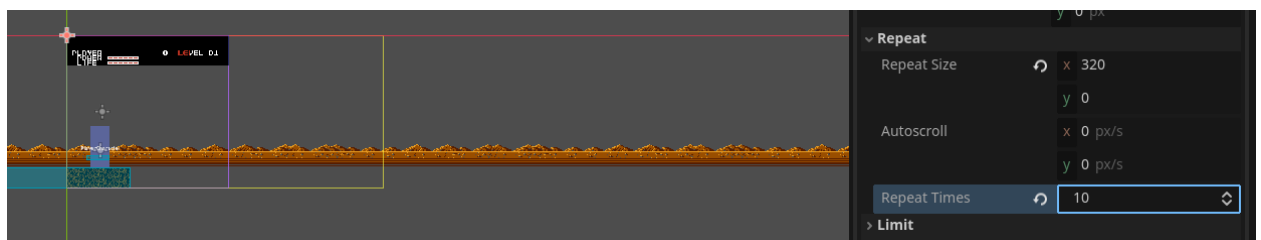
So now the problem about the image not repeating, open the “Repeat” group below and you will notice properties like “Repeat Size” and “Repeat Times”, the ones we need:



Change the Repeat Size X to the width of your image, you can check the image size inside of Godot editor by clicking on the texture inside of the property:



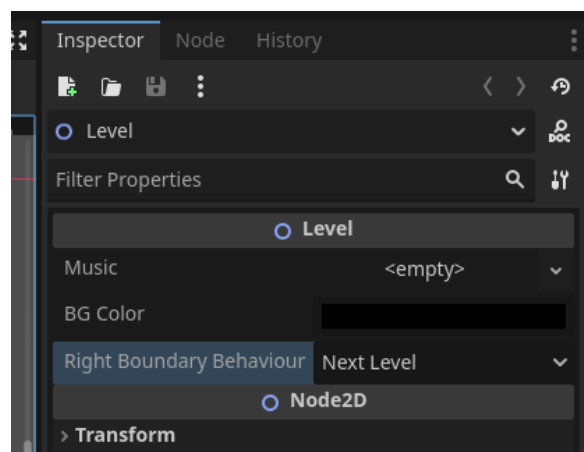
The image was doubled in size! But it's only doubled, it won't repeat past this point. The solution? Increasing "Repeat Times" property!



Here we go! Now you can test the level and see if the settings (primarily the Scroll Scale) suit you.



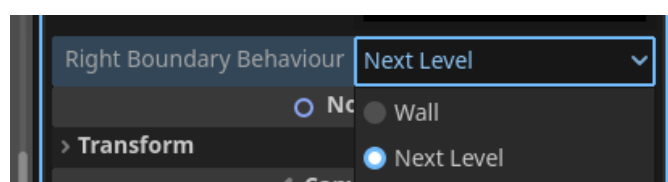
Now let's go over level settings. Click on the “Level” node and you can see it has some custom properties:



There's a music property that determines what music track will be playing in the level, you can drag and drop your music file in here.

BG Color is also pretty self-explanatory, it's the level background color.

There are 2 options for the right boundary behaviour: “Wall” and “Next Level”.



If you choose “Wall”, if the player goes to the end of the level and touches its right boundary it will act as a wall, preventing the player from going further.

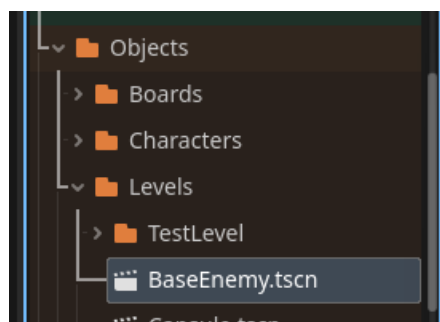
If you choose “Next Level”, the right boundary will act as a level end, sending the player to the planet board.

You might ask, where’s the “Next Planet” behaviour to go to the next planet? You can get to the next planet from the board, by “not moving” on the transition level, more on that in “How to create a new board” tutorial.

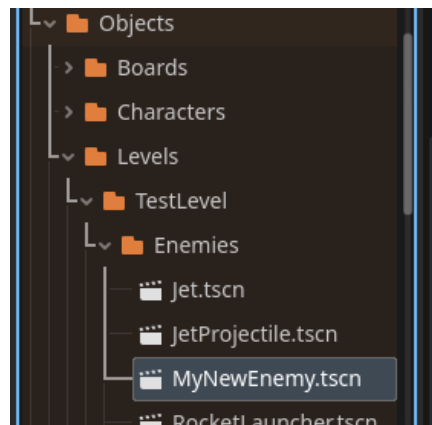
How to create a new enemy

I will create a simple enemy first and then add a small section about projectiles in case you need it!

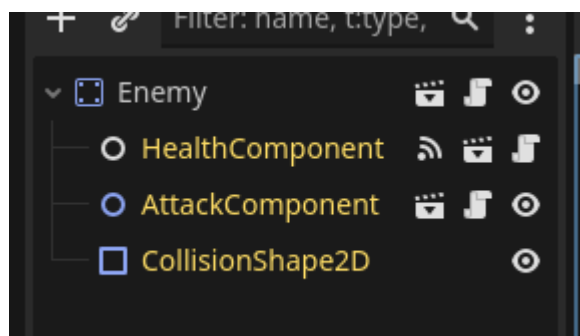
Just like the levels, the enemies also have a base enemy scene:



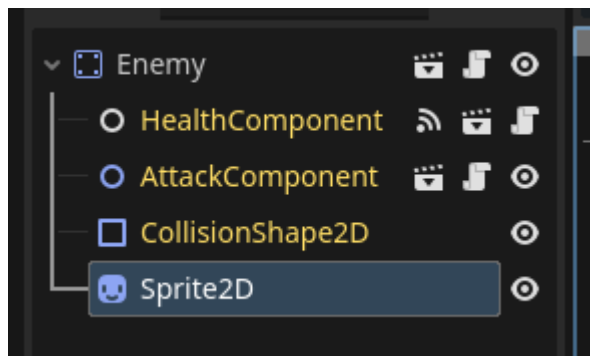
And just like the levels, you can create a new enemy by creating an inherited scene from it.



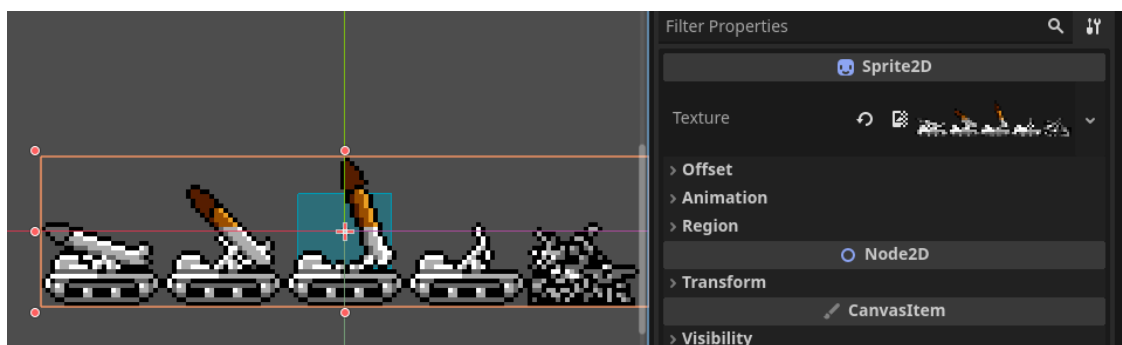
The base enemy scene is pretty barebones, but it has the components required for all enemies:



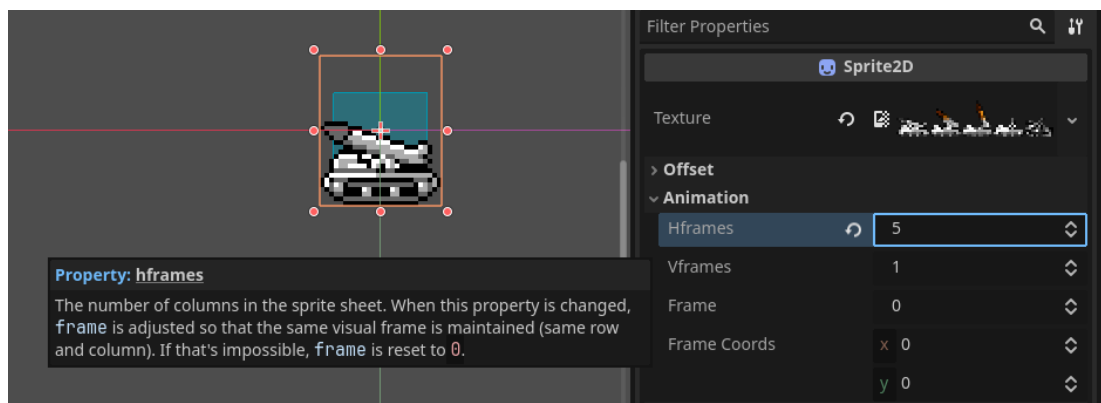
Let's make the enemy visible by adding a sprite to it. You can use Sprite2D, AnimatedSprite2D or other nodes if you want, I will use Sprite2D for this tutorial.



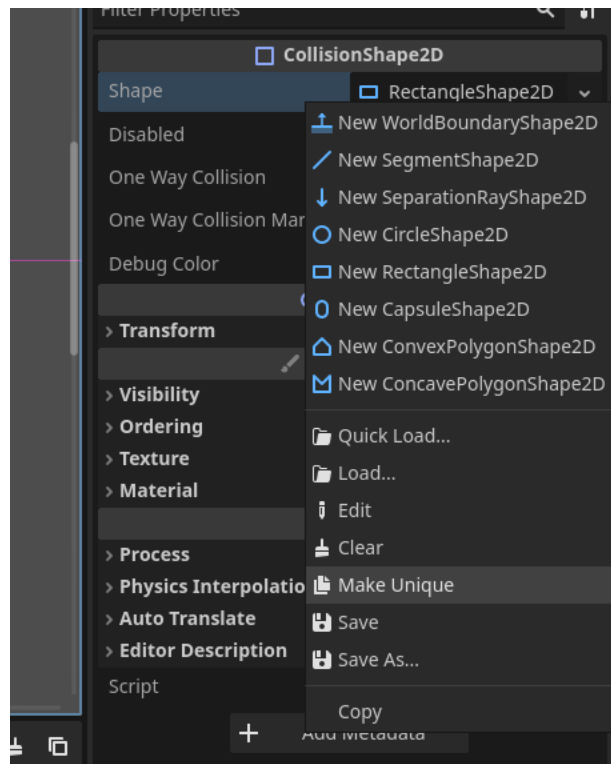
Also, for this tutorial I will use the sprites of an already existing enemy from the framework: the rocket launcher.



I will also need to adjust values like Hframes and Vframes for the spritesheet.



Now the collision shape. First, you need to make it unique, just click on the "CollisionShape2D" node, go to its properties, right click on the shape property and click "Make Unique":

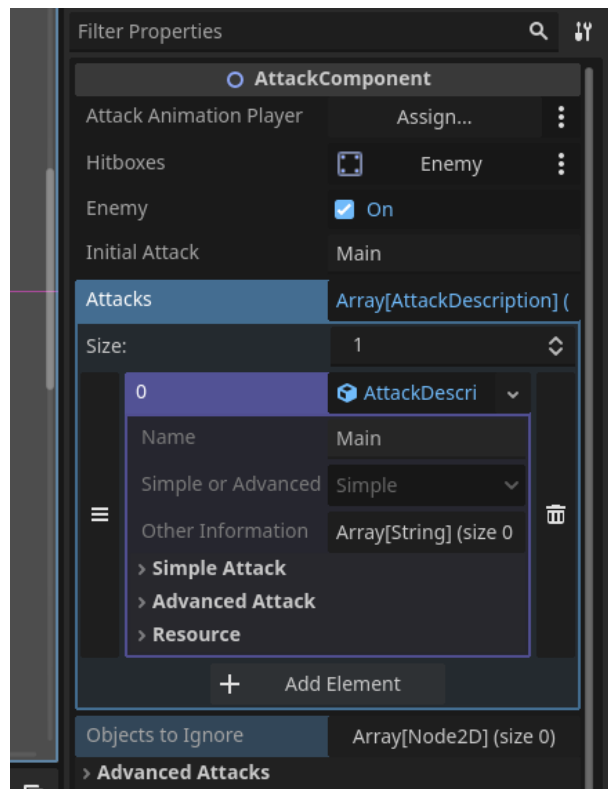


Why is it needed? That's needed in case you want to change the shape but since the inherited scenes also share resources (the shape is a Godot Resource), the changes in the inherited scenes may affect the base scene, so by making the shape unique we basically make a copy of the original shape that will not affect the shape in the base scene.

Now I will reposition the collision shape and/or the sprite (and you can also change the collision shape itself, if needed, it just so happens that the base enemy collision shape matches the rocket launcher).

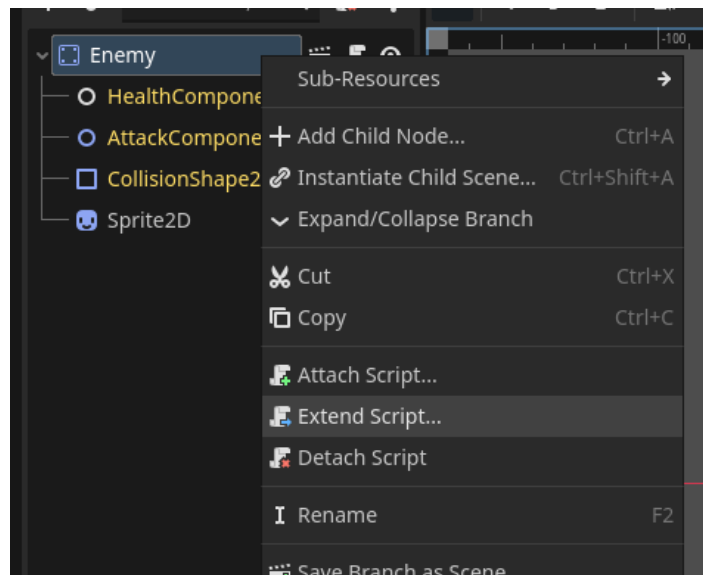


If you need more attacks, you should add them to the attack component (more on that in "How to create a new character" tutorial).

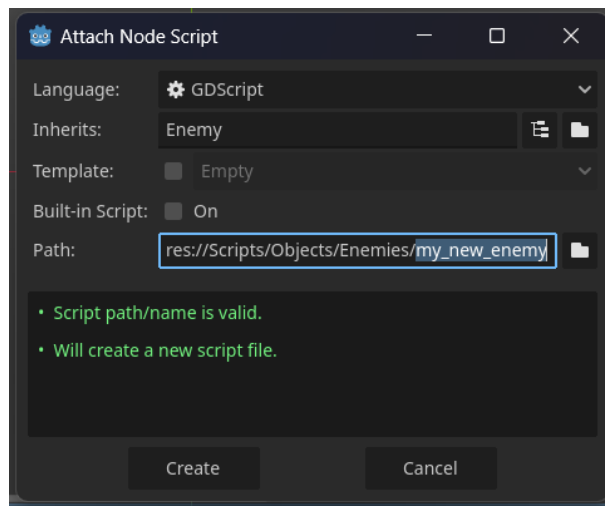


Now if you were to place this new enemy inside of a level it would do nothing except hit the player upon touching it and take damage. So now let's make it do something else as well.

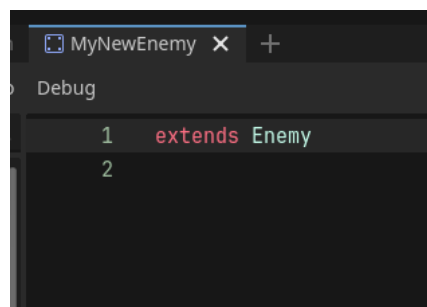
Right click on the "Enemy" node, i.e. the top level node in this scene, and click "Extend Script..."



Place your script somewhere, preferably in "Scripts/Objects/Enemies" folder or in a different folder depending on the planet you're making this enemy for.

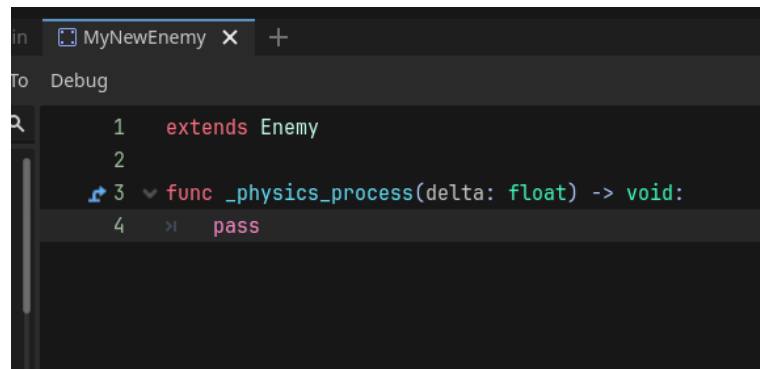


And hit “Create”.

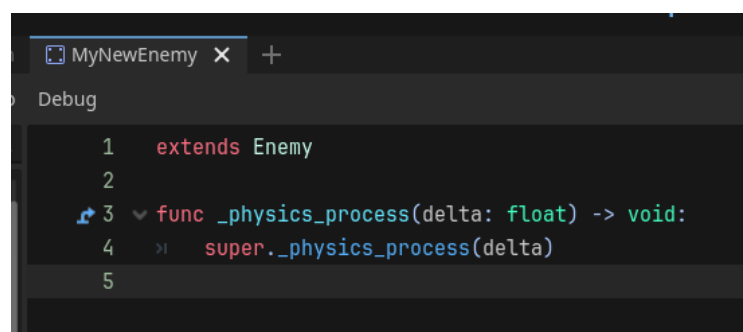


Not sure what I should make it do so I will make it move in a sine wave just as an example.

Just as usual, I will add a function that will be used for physics changes:

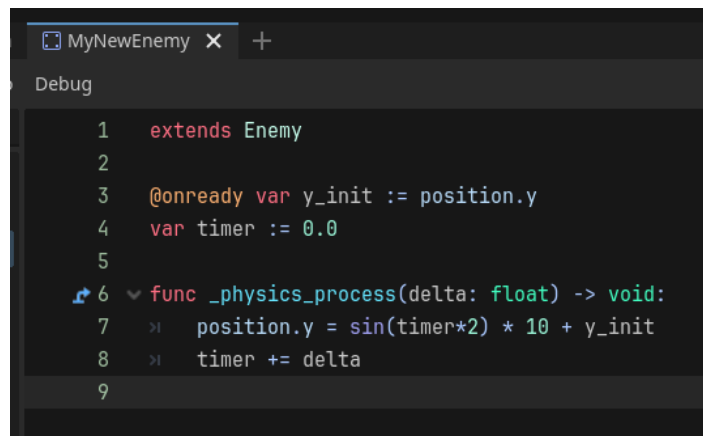


Since there's no “_physics_process” function in the Enemy class, it's fine, but if there was, we would need to call it like this:



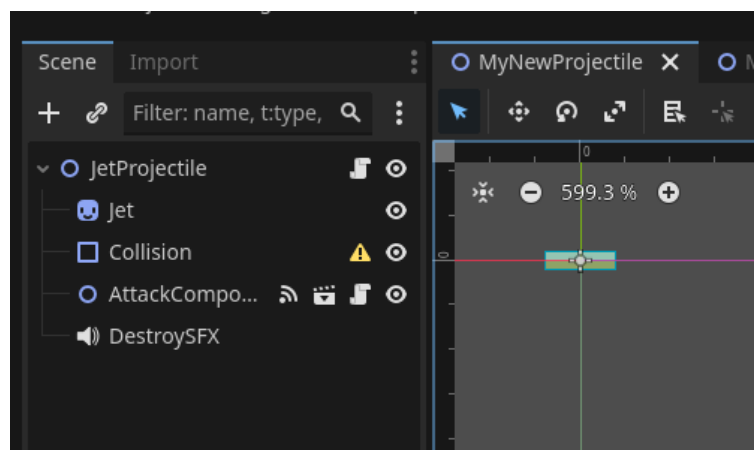
Why? Because if that function existed, there would probably be an important functionality that you would probably not expect to be gone.

So, the sine wave functionality:

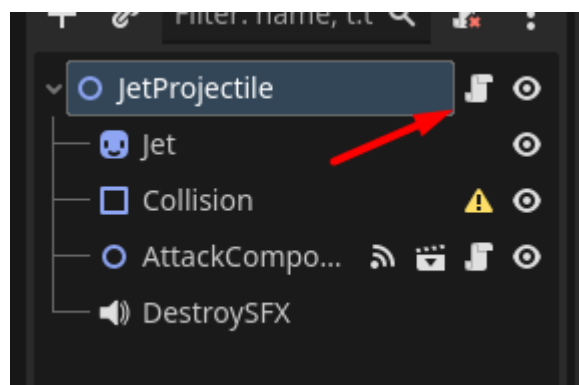


```
1 extends Enemy
2
3 @onready var y_init := position.y
4 var timer := 0.0
5
6 func _physics_process(delta: float) -> void:
7     position.y = sin(timer*2) * 10 + y_init
8     timer += delta
9
```

Now the projectiles. There's no base projectile scene, so we'll need to create our own or duplicate the existing one (they're very simple scenes, so it's not really important to have a base scene and inherit from it). For this tutorial, I will duplicate an already existing projectile, specifically, the jet projectile.



For this tutorial, I will basically duplicate the original jet projectile script so we can edit it a bit. To do that, open the script first by clicking on the scroll icon next to the top level node:

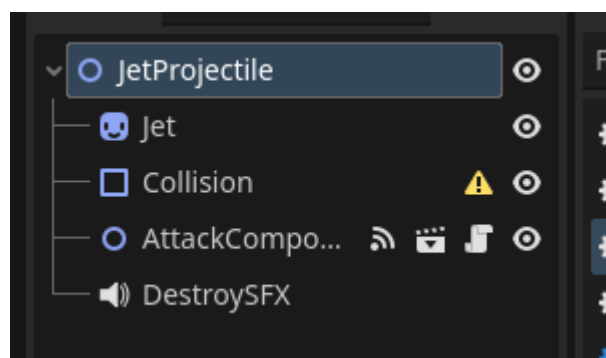
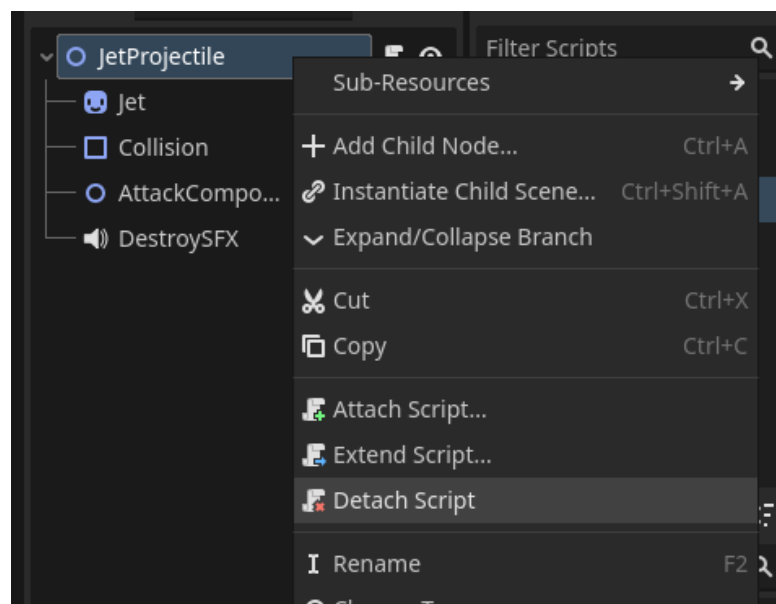


```

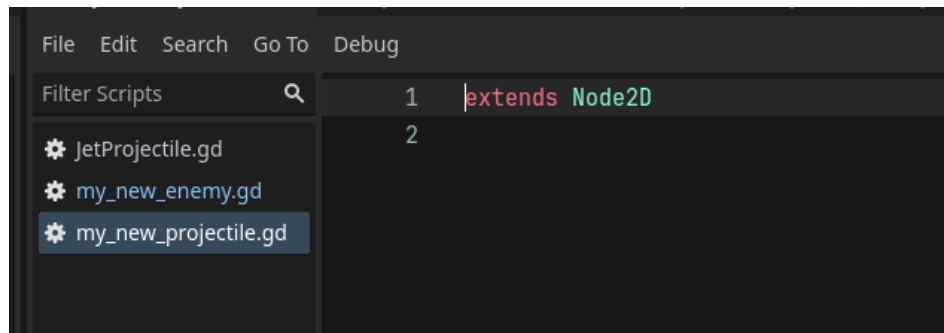
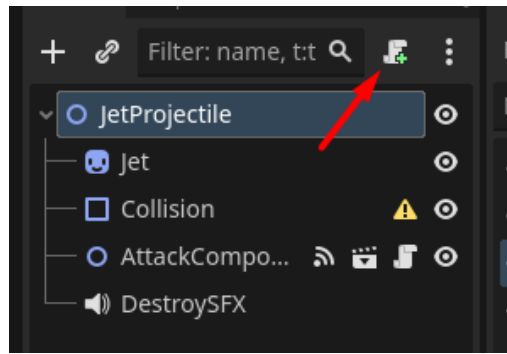
1  extends Node2D
2
3  const VELOCITY := Vector2(-1 * 60, 0.5 * 60)
4  const EXPLOSION := preload("res://Objects/Levels/Explosion.tscn")
5
6  @onready var destroy_sfx: AudioStreamPlayer = $DestroySFX
7  @onready var attack_component: AttackComponent = $AttackComponent
8
9  func _process(delta: float) -> void:
10     position += VELOCITY * delta
11
12  func _on_attack_component_attacked(_body: Node2D, _amount: float) -> void:
13     var explosion := EXPLOSION.instantiate()
14     explosion.global_position = global_position
15
16     destroy_sfx.play()
17     destroy_sfx.reparent(get_parent())
18     destroy_sfx.finished.connect(func() -> void: destroy_sfx.queue_free())
19     add_sibling(explosion)
20     queue_free()
21

```

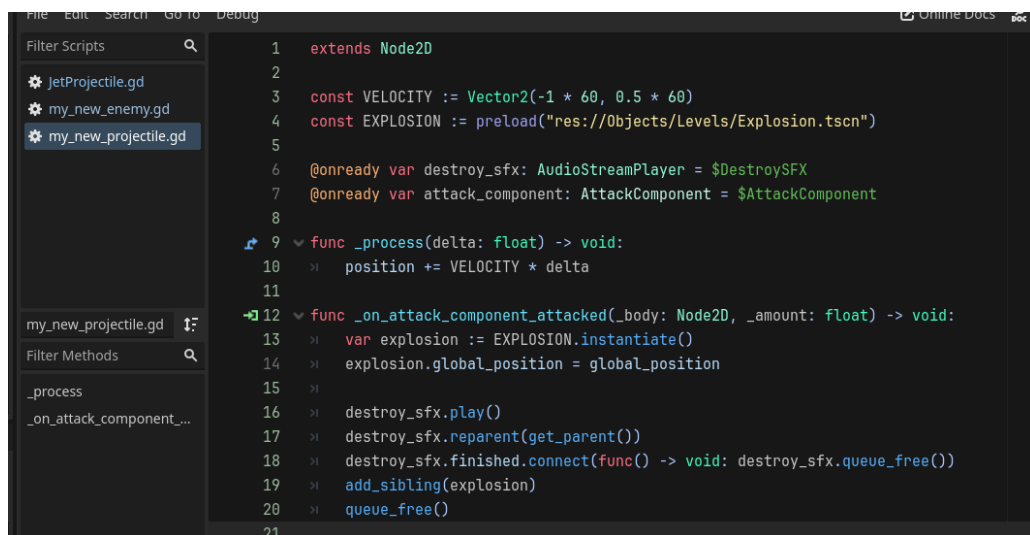
Select everything (you can use a keyboard shortcut for that: Ctrl+A) and copy.
Now detach the existing script:



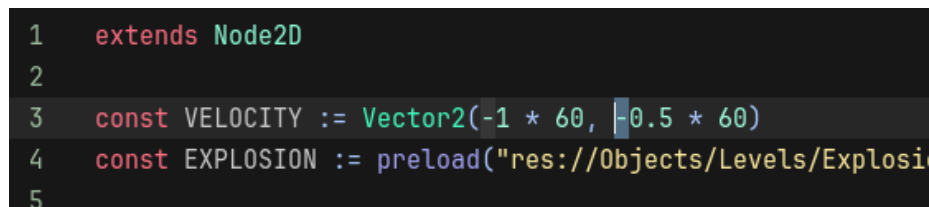
Create a new one:



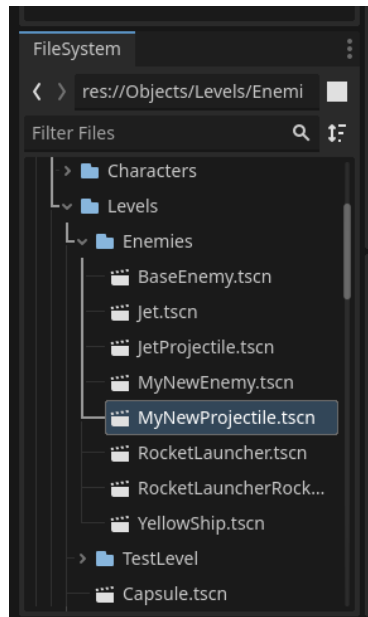
Remove the “extends Node2D” line and past the copied script here:



Nice, so now let’s change it a bit, for example, we can make it go up instead of down by changing the “VELOCITY” constant at the top of the script like this:



Now let’s make our enemy shoot this projectile. First, let’s load it in the script. You can do that in the built in editor very easily this way: first, find the projectile scene in the FileSystem dock:

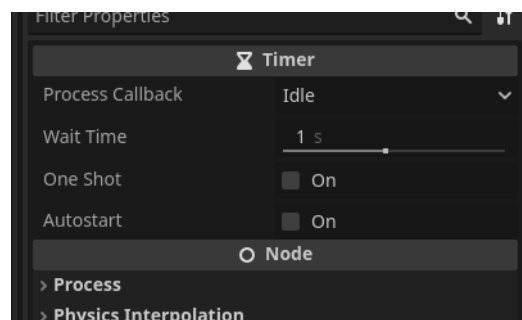
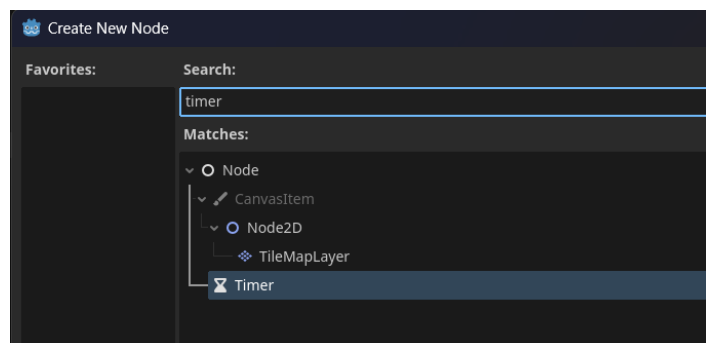


Drag and drop it somewhere in your script (preferably at the top) while holding Ctrl key:

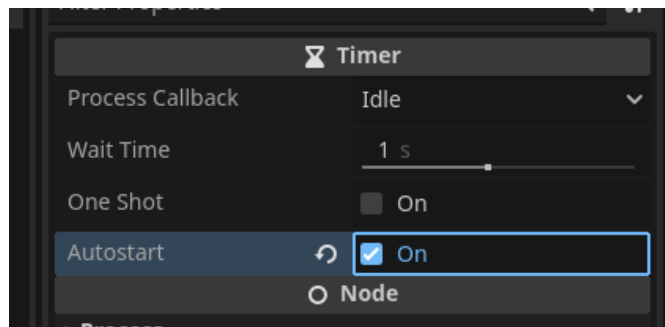
```
bug
1 extends Enemy
2 const MY_NEW_PROJECTILE = preload("res://Objects/Levels/Enemies/MyNewProjectile.tscn")
3 @onready var y_init := position.y
4 var timer := 0.0
5
```

Wow, it just magically added the exact line that loads the scene inside of the script, nice!

Now let's add a timer node that will fire every 1 second, and later we will connect the projectile creation to the timer.



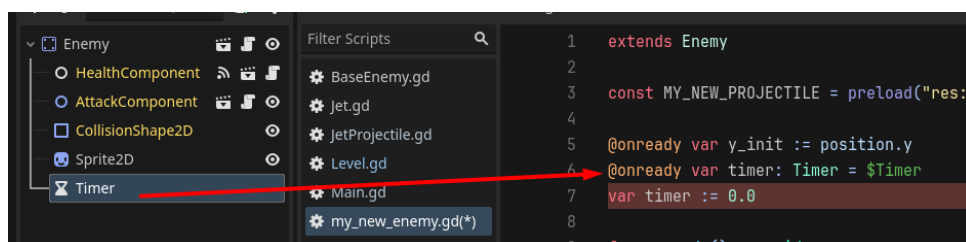
1 second is already the default value, so we can leave it like that, so now let's just check the "Autostart" property so it... starts automatically.



Now we can connect to the timer inside the code. First, let's create the "_ready" function.

```
5 @onready var y_init := position.y
6 var timer := 0.0
7
8 func _ready() -> void:
9     pass
10
11 func _physics_process(delta: float)
12     position.y = sin(timer*2) * 10 +
```

Now let's drag and drop the timer node into the script while holding Ctrl key:



This also works for nodes! It added the line that creates a new variable that references the node that was dragged, but now we have a name conflict, so we can change the timer node variable name a bit:

```
4
5 @onready var y_init := position.y
6 @onready var timer_node: Timer = $Timer
7 var timer := 0.0
8
```

Now let's create a placeholder function for the projectiles.

```

11
12 ▾ func _physics_process(delta: float) ->
13     position.y = sin(timer*2) * 10 + y
14     timer += delta
15
16 ▾ func shoot_projectile() -> void:
17     pass
18

```

And now we can connect it to the timer signal in the “_ready” function like this:

```

8
9 ▾ func _ready() -> void:
10     timer_node.timeout.connect(shoot_projectile)
11

```

We can copy the jet code that shoots projectiles, so open the jet scene and open its script.

```

34     animation_player.play("flying_right")
35
36     State.MOVING_RIGHT:
37         velocity.x += 0.05 * 60 * 60 * delta
38         if not launched_projectile and absf(velocity.x) < 0.04 * 60:
39             var projectile := JET_PROJECTILE.instantiate()
40             projectile.position = position
41             add_sibling(projectile)
42             projectile.attack_component.objects_to_ignore.append(self)
43             launched_projectile = true
44
45     position += velocity * delta
46
47 ▾ func _on_attack_component_attacked(_body: Node2D, _amount: float) -> void:

```

There's the code we need. Let's, ahem, *borrow* it and add it to our code:

```

16 ▾ func shoot_projectile() -> void:
17     var projectile := JET_PROJECTILE.instantiate()
18     projectile.position = position
19     add_sibling(projectile)
20     projectile.attack_component.objects_to_ignore.append(self)
21

```

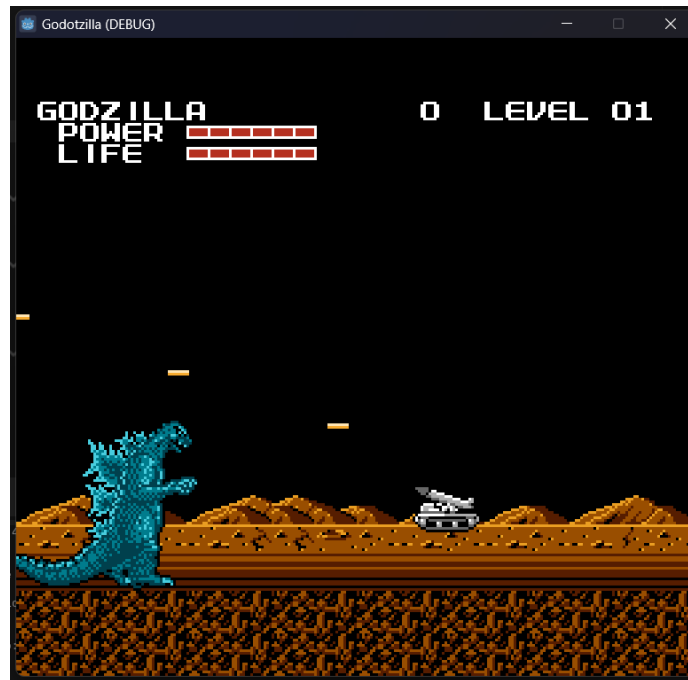
...while also using the correct names:

```

16 ▾ func shoot_projectile() -> void:
17     var projectile := MY_NEW_PROJECTILE.instantiate()
18     projectile.position = position
19     add_sibling(projectile)
20     projectile.attack_component.objects_to_ignore.append(self)
21

```

Nice. Now if you place it inside a level, you will see it slowly moves up and down and shoots projectiles every second.



How to create a new character state

So, you want to expand the functionality of an existing character or for your own custom character by creating a new state the character can be in? I can help you with that!

So first of all, you should know if what you want to create should be a new separate state:

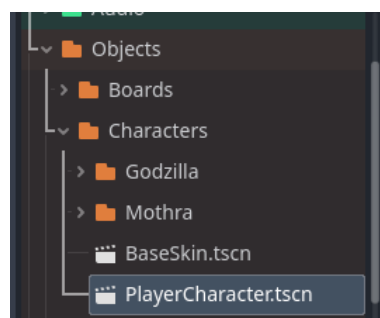
Godzilla becoming frozen? New state.

Godzilla getting poisoned? It can be either a new state or some additional code in the PlayerCharacter class.

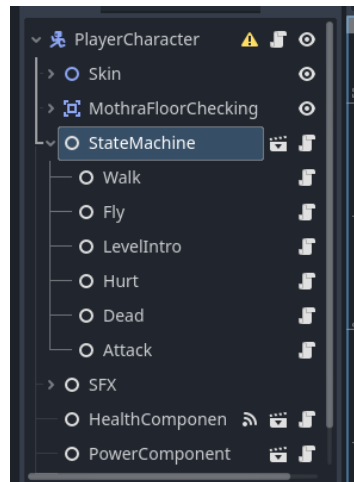
Godzilla getting a new attack? Not a new state, but rather a new attack should do it (check out the How to create a new character part 2, it can also be used for existing characters if you want).

So now if you think you should create a new state for your functionality, let's create one!

Open the PlayerCharacter scene:

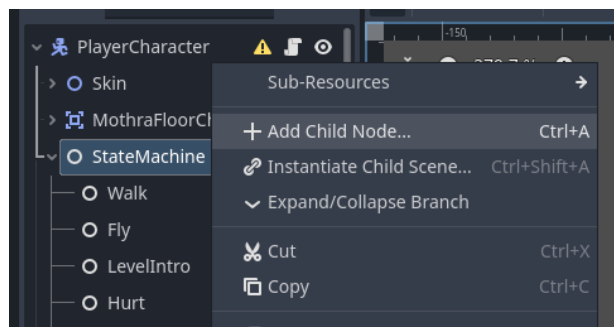


You can see it has a “state machine” inside:

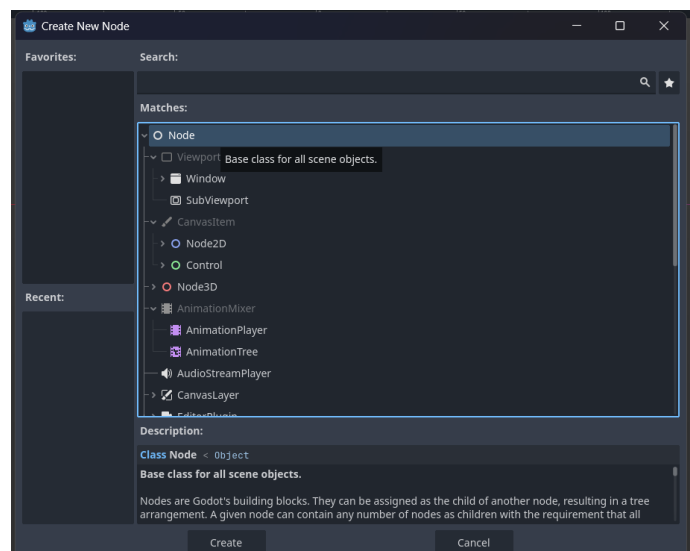


So, what is a “state machine”? For that, you can check out the “Framework overview” tutorial :D

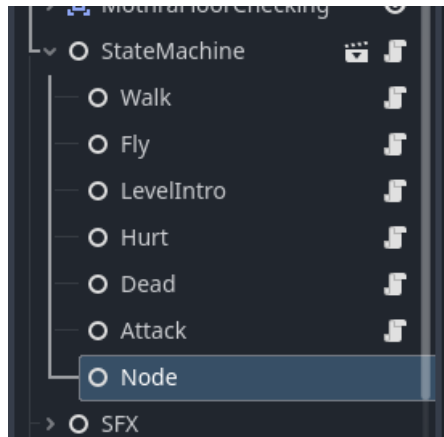
Now let’s create a new state. Right click the “StateMachine” node and then click “Add Child Node...”:



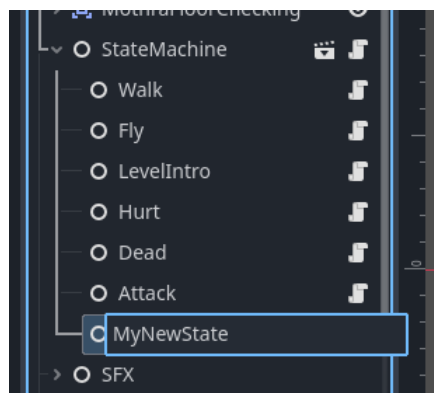
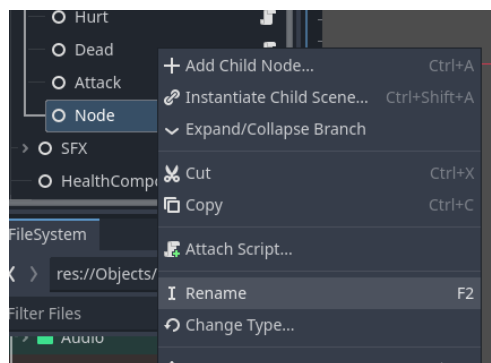
You should select the basic Node type for it since it doesn’t need to do anything other than describing a character’s behaviour:



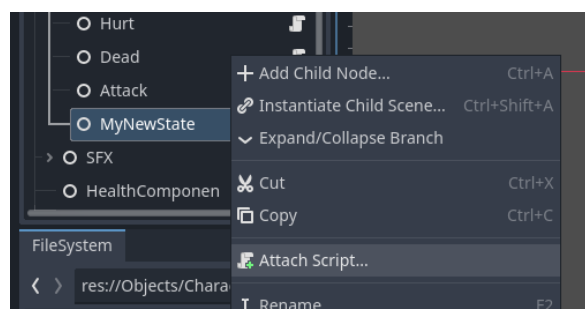
A new node was added:

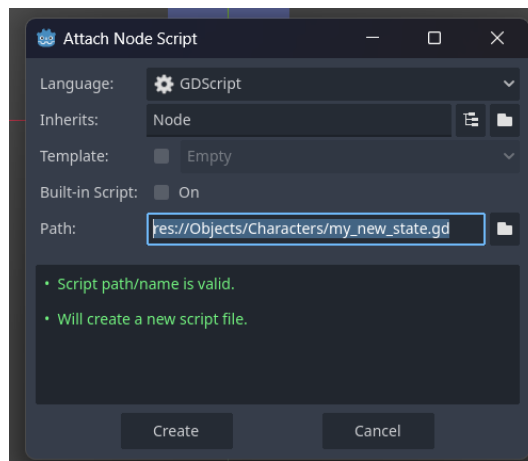


Rename it:

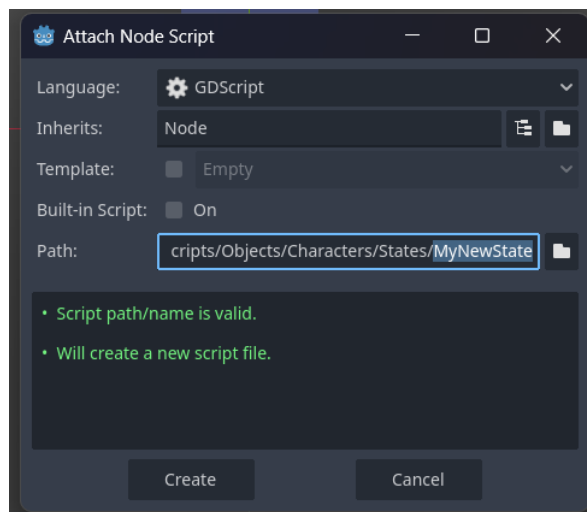


Now you can see that other states have scripts attached to them and our new state doesn't, let's create one!

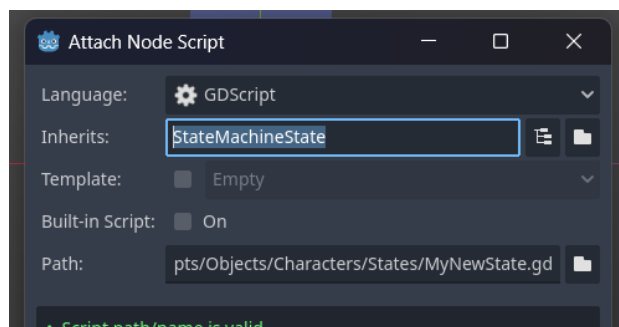




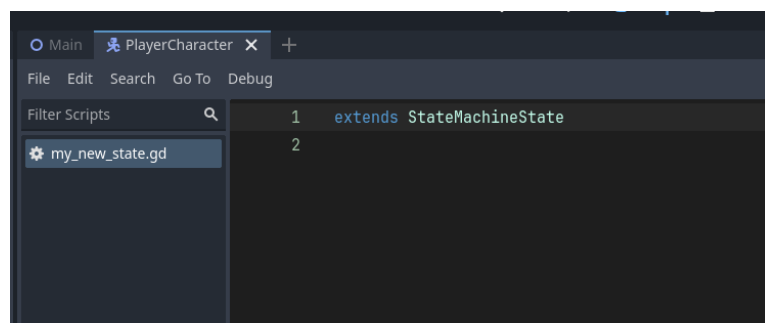
Place it among other states, in the “Scripts/Objects/Characters/States” folder:



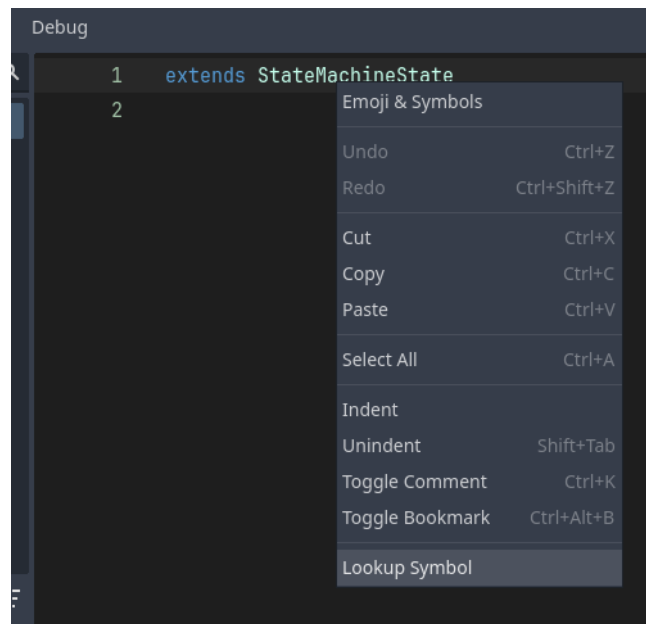
Now let’s make it inherit the class that all states should inherit from. Replace “Node” text here to “StateMachineState” like this:



Now click “Create”. A new script window should appear:



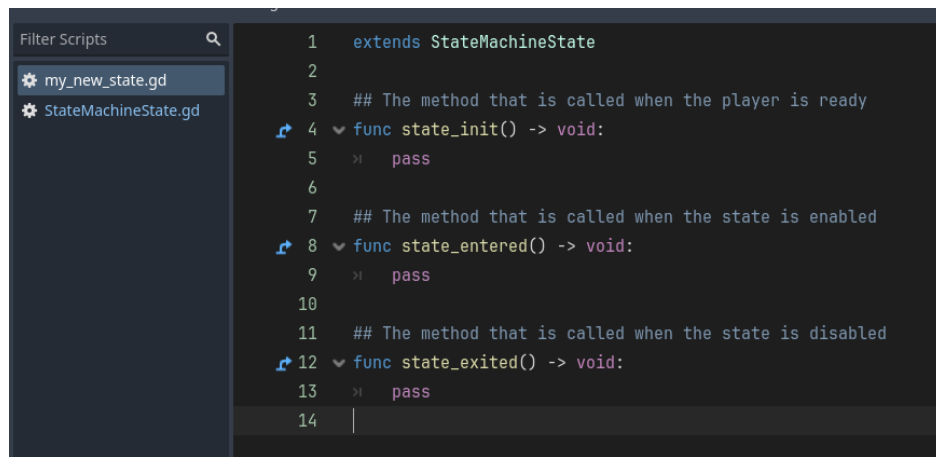
Now let's see what code we can put in here. Right click the "StateMachineState" text after "extends" and then click on "Lookup Symbol":



The code for that class appears:

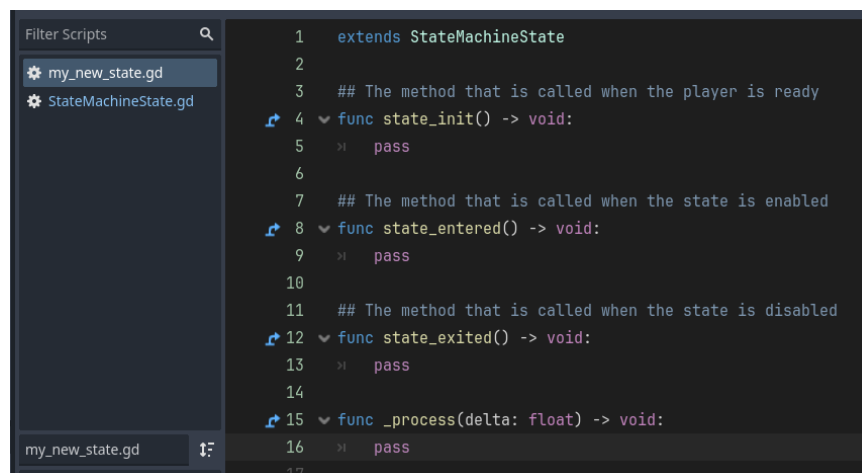
```
1  class_name StateMachineState extends Node
2
3  @onready var parent: Node = $"../.."
4
5  ## The method that is called when the player is ready
6  func state_init() -> void:
7      pass
8
9  ## The method that is called when the state is enabled
10 func state_entered() -> void:
11     pass
12
13 ## The method that is called when the state is disabled
14 func state_exited() -> void:
15     pass
16
17 func enable() -> void:
18     set_process(true)
19     set_physics_process(true)
20     set_process_input(true)
21
22 func disable() -> void:
23     set_process(false)
24     set_physics_process(false)
25     set_process_input(false)
26
```

So there are 3 empty methods, and we can change these methods in our new script:



```
1 extends StateMachineState
2
3 ## The method that is called when the player is ready
4 func state_init() -> void:
5     pass
6
7 ## The method that is called when the state is enabled
8 func state_entered() -> void:
9     pass
10
11 ## The method that is called when the state is disabled
12 func state_exited() -> void:
13     pass
14
```

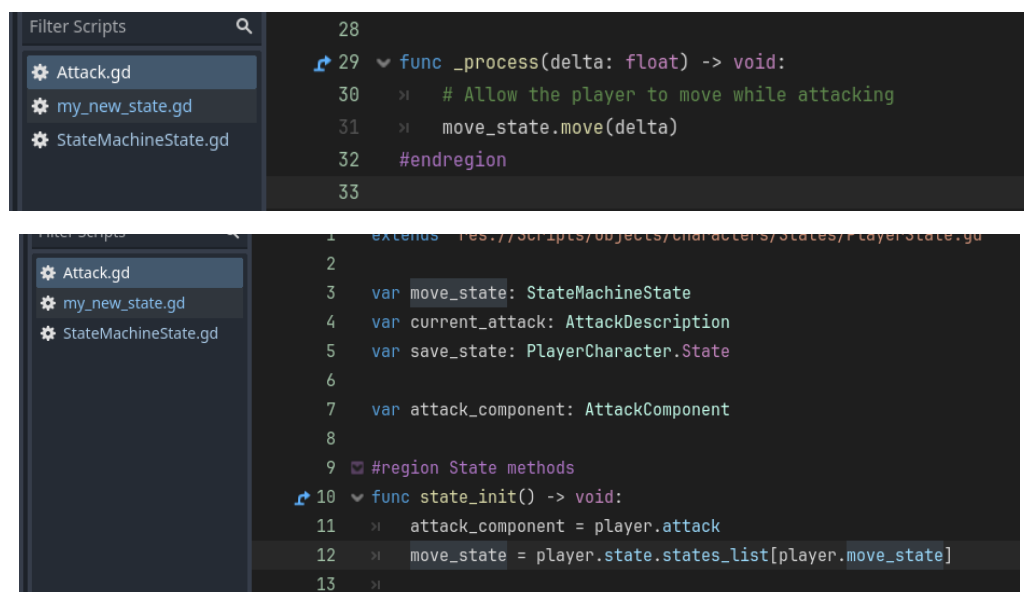
We can also of course replace Godot Node’s built-in methods like “_process” if we need to run some code every frame:



```
1 extends StateMachineState
2
3 ## The method that is called when the player is ready
4 func state_init() -> void:
5     pass
6
7 ## The method that is called when the state is enabled
8 func state_entered() -> void:
9     pass
10
11 ## The method that is called when the state is disabled
12 func state_exited() -> void:
13     pass
14
15 func _process(delta: float) -> void:
16     pass
17
```

You can see what each method means by looking at the comment above its name.

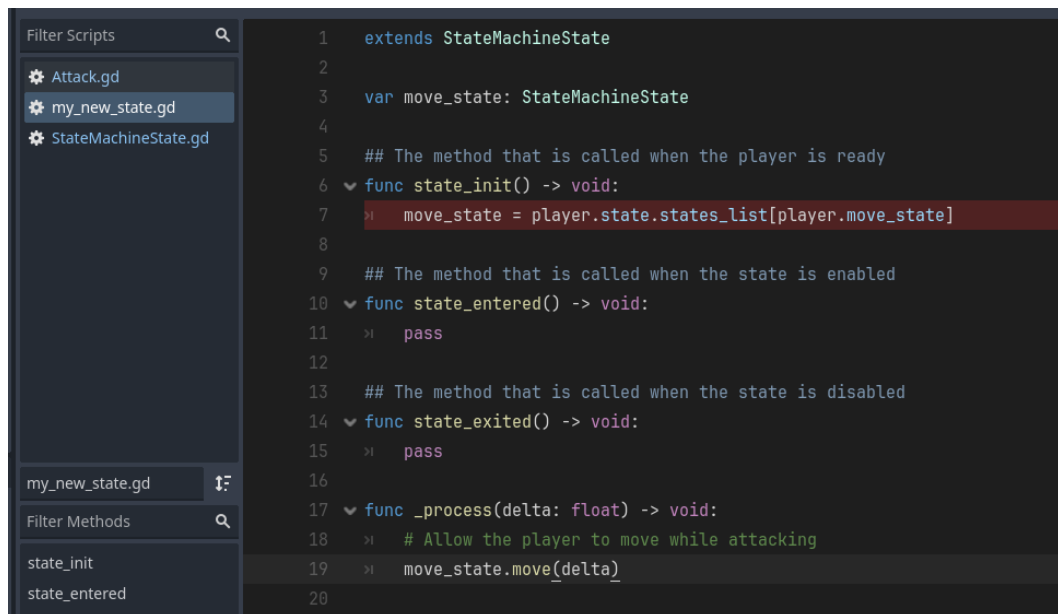
If you want your character to move while its in this new state, you can see how other states do that, like the attacking state:



```
28
29 func _process(delta: float) -> void:
30     # Allow the player to move while attacking
31     move_state.move(delta)
32 #endregion
33
```

```
1 extends Res://Scripts/objects/characters/states/PlayerState.gd
2
3 var move_state: StateMachineState
4 var current_attack: AttackDescription
5 var save_state: PlayerCharacter.State
6
7 var attack_component: AttackComponent
8
9 #region State methods
10 func state_init() -> void:
11     attack_component = player.attack
12     move_state = player.state.states_list[player.move_state]
13
```

Let’s borrow some code from here:



```
1 extends StateMachineState
2
3 var move_state: StateMachineState
4
5 ## The method that is called when the player is ready
6 func state_init() -> void:
7     move_state = player.state.states_list[player.move_state]
8
9 ## The method that is called when the state is enabled
10 func state_entered() -> void:
11     pass
12
13 ## The method that is called when the state is disabled
14 func state_exited() -> void:
15     pass
16
17 func _process(delta: float) -> void:
18     # Allow the player to move while attacking
19     move_state.move(delta)
20
```

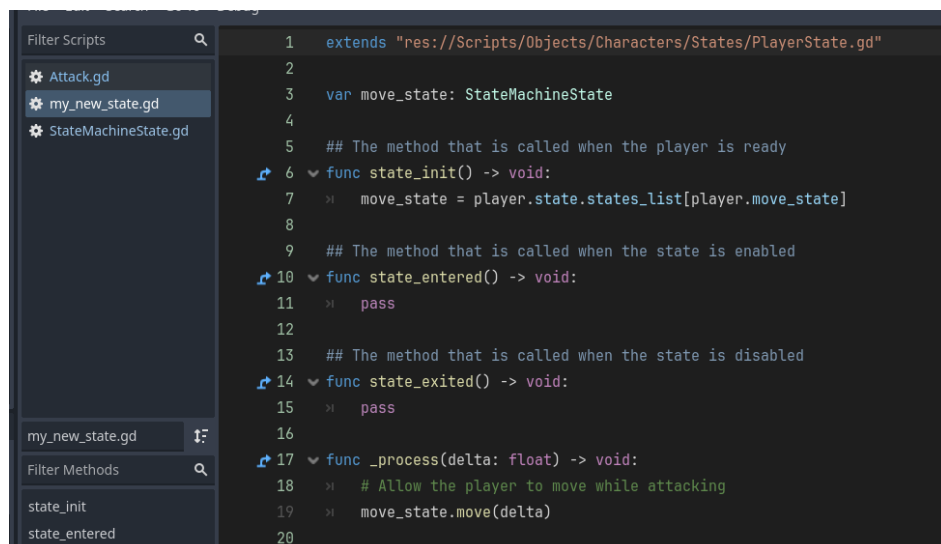
Something is not right here, Godot can't find the "player" variable.

Oh, right, we're making a character state, so we should instead make the class inherit the player state class, like other player states:



```
1 extends "res://Scripts/Objects/Characters/States/PlayerState.gd"
2
3 var move_state: StateMachineState
4 var current_attack: AttackDescription
5 var save_state: PlayerCharacter.State
6
7 var attack_component: AttackComponent
8
```

Let's copy this line and paste it into our class:

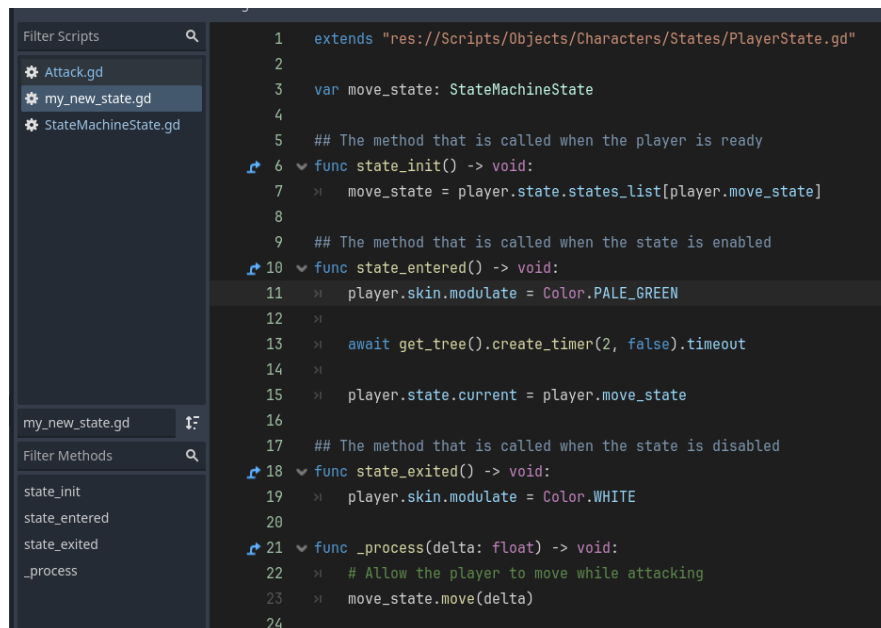


```
1 extends "res://Scripts/Objects/Characters/States/PlayerState.gd"
2
3 var move_state: StateMachineState
4
5 ## The method that is called when the player is ready
6 func state_init() -> void:
7     move_state = player.state.states_list[player.move_state]
8
9 ## The method that is called when the state is enabled
10 func state_entered() -> void:
11     pass
12
13 ## The method that is called when the state is disabled
14 func state_exited() -> void:
15     pass
16
17 func _process(delta: float) -> void:
18     # Allow the player to move while attacking
19     move_state.move(delta)
20
```

Ah, much better!

Now let's make the player do something in this state.

For this example, I will create a “poisoned” state, so I will change the player’s color to green and after some time it will return back into its original state.



The screenshot shows the Godot 4.0 IDE. On the left, the 'Filter Scripts' panel lists 'Attack.gd', 'my_new_state.gd', and 'StateMachineState.gd'. Below it, the 'Filter Methods' panel lists 'state_init', 'state_entered', 'state_exited', and '_process'. The main editor displays the code for 'my_new_state.gd'.

```
1 extends "res://Scripts/Objects/Characters/States/PlayerState.gd"
2
3 var move_state: StateMachineState
4
5 ## The method that is called when the player is ready
6 func state_init() -> void:
7     move_state = player.state.states_list[player.move_state]
8
9 ## The method that is called when the state is enabled
10 func state_entered() -> void:
11     player.skin.modulate = Color.PALE_GREEN
12
13     await get_tree().create_timer(2, false).timeout
14
15     player.state.current = player.move_state
16
17 ## The method that is called when the state is disabled
18 func state_exited() -> void:
19     player.skin.modulate = Color.WHITE
20
21 func _process(delta: float) -> void:
22     # Allow the player to move while attacking
23     move_state.move(delta)
24
```

Let’s break down this code a bit.

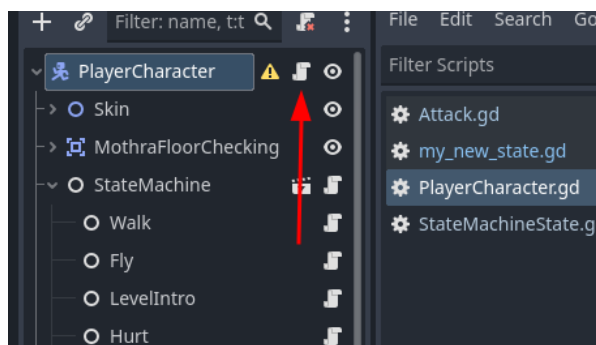
In the “state_entered” function I added “player.skin.modulate = Color.PALE_GREEN”, this line changes the player’s skin color to be a bit more green;

“await get_tree().create_timer(2, false).timeout” waits 2 seconds before running the next code;

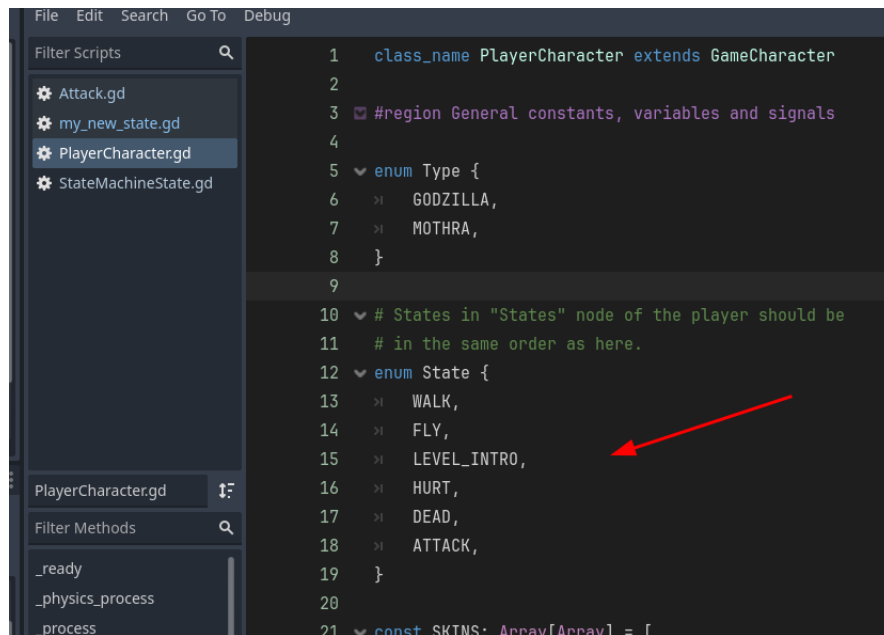
“player.state.current = player.move_state” changes the code back to the default character moving state.

In the “state_exited” function I added “player.skin.modulate = Color.WHITE”, which changes the player skin back to its original color.

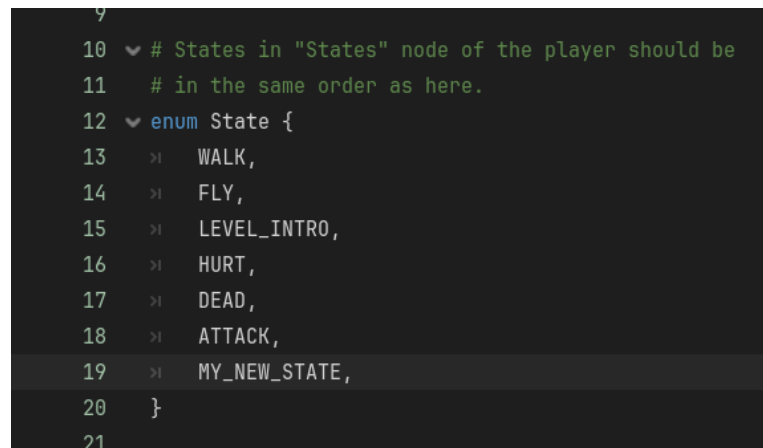
Now let’s create a new ID for our new state. Open the PlayerCharacter script by pressing this button:



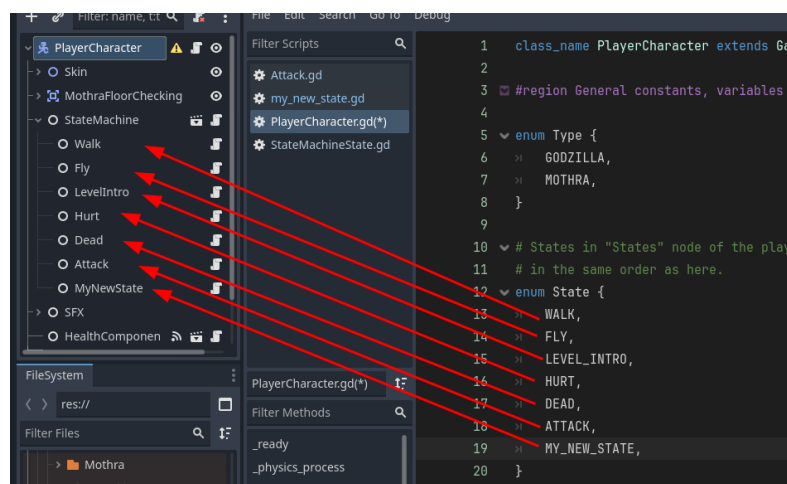
At the top of the file you’ll see a list of action names:



That's where we add our new action!



But, just like what the comment at the top says, make sure the states in this list and in the node tree are in the same order:



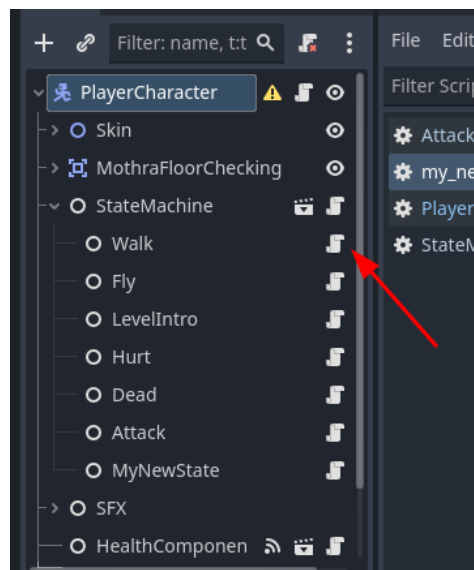
What would happen if they were not? Well, the framework changes the character's state by using IDs of the nodes inside the state machine node, so "WALK" inside the enum corresponds to the first children of the state machine, so the "Walk" state,

if the “WALK” value was not the first one, and, for example, “FLY” was the first, then by setting the character’s state to “FLY” it would go into the “Walk” state (because the id of “FLY” is the first one and the id of the “Walk” state is also the first one).

So now how do we change the state to our new state? Well, we already know how to change the character state:

```
11 > player.skin.modulate = Color.FX_GREEN
12 >
13 > await get_tree().create_timer(2, false).timeout
14 >
15 > player.state.current = player.move_state
16
17 ## The method that is called when the state is disabled
18 func state_exited() -> void:
19     player.skin.modulate = Color.WHITE
```

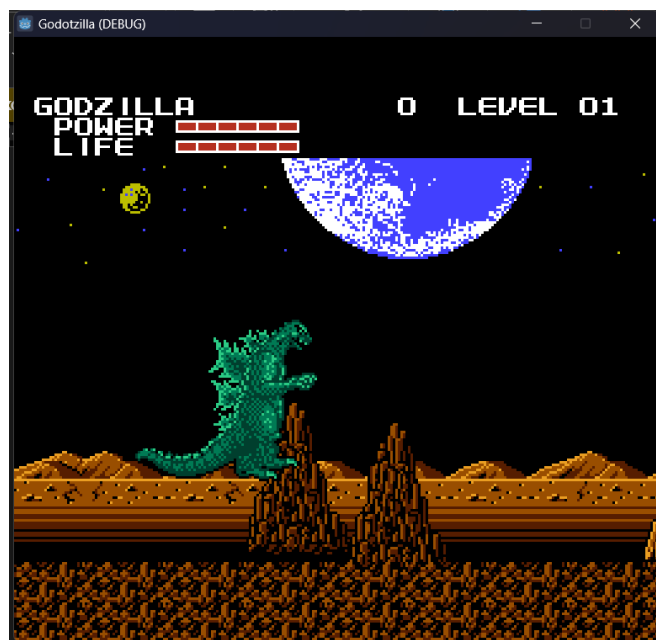
We can use this code to make the character go into our new state. But how should the character go into this state? That’s up to you to decide, but for this example I will make the character go into this state whenever the Up button is pressed while in walking state. So, for that I would go into the code of the Walking state:



And then I would add the needed code inside the “_process” function (the check for pressing that button should be done every frame, since I need the character to go into my new state at any time while walking):

```
19
20 func _process(delta: float) -> void:
21     move(delta)
22     player.skin.walk_process()
23
24     if Input.is_action_just_pressed("Up"):
25         player.state.current = PlayerCharacter.State.MY_NEW_STATE
26
27 func _move(delta: float) -> void:
```

Now let's try it in-game!



It works!